give it

A **linked list** is a linear data structure where elements (nodes) are not stored contiguously in memory but linked via pointers, with each node containing data and a reference to the next node (and optionally previous for doubly-linked).[1] [2] [3]

Linked List vs Array/List

Use linked lists over arrays when frequent insertions/deletions are needed at arbitrary positions, as arrays require O(n) shifts while linked lists do this in O(1) with a pointer. Arrays excel in random access (O(1)) and cache efficiency, but linked lists save space for sparse/dynamic data and avoid resizing overhead.[2] [4] [1]

Aspect	Linked List	Array/List	
Insertion/Deletion	O(1) at known position	O(n) due to shifting	
Random Access	O(n) traversal	O(1) indexing	
Memory	Dynamic, no fixed size	Fixed or resize costly	
Extra Space	O(n) for pointers	Contiguous block	[1] [2]

Common Methods

- **Insertion**: At head (O(1)), tail (O(1) with tail pointer), or position (O(n)).
- **Deletion**: By value/position (O(n) search + O(1) unlink).
- **Traversal/Search**: O(n) iterate via next pointers.
- **Reverse**: O(n) by flipping pointers.[3] [5] [6]

Key Algorithms

- Fast-slow pointers (e.g., cycle detection, middle node).
- Merge two sorted lists (O(n+m)).
- Reverse in groups of k.
- Remove nth node from end.[5] [1]

Top Interview Questions

- Reverse a linked list.
- Detect/remove cycle (Floyd's tortoise-hare).
- Merge two sorted lists.
- Find middle node / palindrome check.
- Add two numbers (digits as nodes).[1] [2]

⁂

1. https://www.geeksforgeeks.org/dsa/linked-list-data-structure/
2. https://en.wikipedia.org/wiki/Linked_list
3. https://www.programiz.com/dsa/linked-list
4. https://www.w3schools.com/dsa/dsa_theory_linkedlists.php
5. https://www.tutorialspoint.com/data_structures_algorithms/linked_list_algorithms.htm
6. https://byjus.com/gate/linked-list-notes/